COLLEGE CODE : 9513

COLLEGE NAME : JAYARAJ ANNAPACKIAM CSI COLLEGE OF ENGINEERING

DEPARTMENT : COMPUTER SCIENCEAND ENGINEERING

STUDENT NM-ID : 582ae44a3f49c60bdbf4976fa1bbea20

ROLL NO : 951323104050

DATE : 4.10.2025

Completed the project named as

Phase 5 TECHNOLOGY

PROJECT NAME : Live Weather Dashboard

SUBMITTED BY,

NAME : SRI MITHUNA.M
MOBILE NO: 8973702633

Live Weather Dashboard

Final Demo Walkthrough

In our project, **Live Weather Dashboard**, we built a website that shows **real-time weather details** for any city in the world.
The user just types a city name, and our system instantly shows the **current weather** along with a **3-day forecast**.

Final Demo – What We Did

For our project, we built a **Live Weather Dashboard** that lets users check the weather of any city in real-time. We added a **search bar** where users can type a city name, and the dashboard fetches **current weather data** from the OpenWeatherMap API. This includes **temperature, humidity, wind speed, weather description, and a weather icon**.

We also included a **3-day weather forecast**, showing the **minimum and maximum temperatures and icons** for each day, making it easy to plan ahead. To handle mistakes, we added **error messages** for invalid city names so the dashboard doesn't crash.

We formatted the **date in day/month/year format** for better readability, and made sure the dashboard is **responsive**, so it works well on both desktop and mobile screens. Finally, we made some **small styling improvements** to make the interface clean, simple, and user-friendly.

Project Report – Live Weather Dashboard

1. Introduction

The **Live Weather Dashboard** is a web application that allows users to check the weather of any city in real-time. It provides **current weather details** as well as a **3-day forecast** using the **OpenWeatherMap API**. The dashboard is designed to be **clean, simple, and responsive**, making it easy for users to quickly check weather updates.

---

2. Objectives

- Fetch real-time weather data for any city.
- Display **current weather information**: temperature, humidity, wind speed, weather description, and icon.
- Show a **3-day weather forecast** with min/max temperatures and icons.
- Handle **invalid city input** gracefully.
- Ensure the dashboard is **responsive** for both desktop and mobile devices.

- Provide a **user-friendly and visually clear interface**.

---

3. What We Did

- Built the **Live Weather Dashboard** using **React.js**.
- Added a **search bar** for users to enter city names.
- Fetched **current weather data** from the OpenWeatherMap API.
- Displayed **temperature, humidity, wind speed, weather description, and icon** dynamically.
- Created a **3-day weather forecast**, showing **min/max temperatures and icons**.
- Implemented **error handling** for invalid city input.
- Formatted **dates as DD/MM/YYYY** for clarity.
- Made the dashboard **responsive** for desktop and mobile.
- Applied **small styling improvements** for a clean, user-friendly interface.

---

4. Challenges & Solutions

Challenge	Solution
Invalid city input	Displayed friendly error message using conditional rendering
Forecast data too detailed	Grouped 3-hour interval data into a 3-day forecast
Responsive layout	Adjusted CSS for mobile and desktop screens
Date formatting	Converted API timestamps to DD/MM/YYYY

---

5. Conclusion

The Live Weather Dashboard provides **real-time weather updates** and a **3-day forecast** in a **simple and easy-to-use interface**. It handles errors gracefully, works on all devices, and presents weather information clearly and efficiently. The dashboard is **responsive, visually clean, and user-friendly**, making it useful for anyone who wants quick access to weather information.

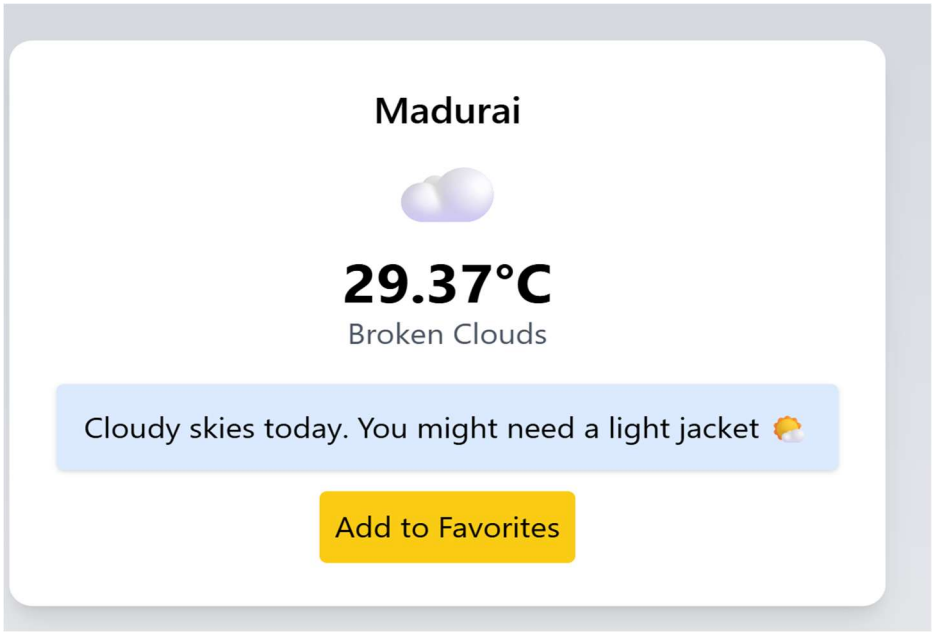
4. Screenshots / API Documentation

Screenshots

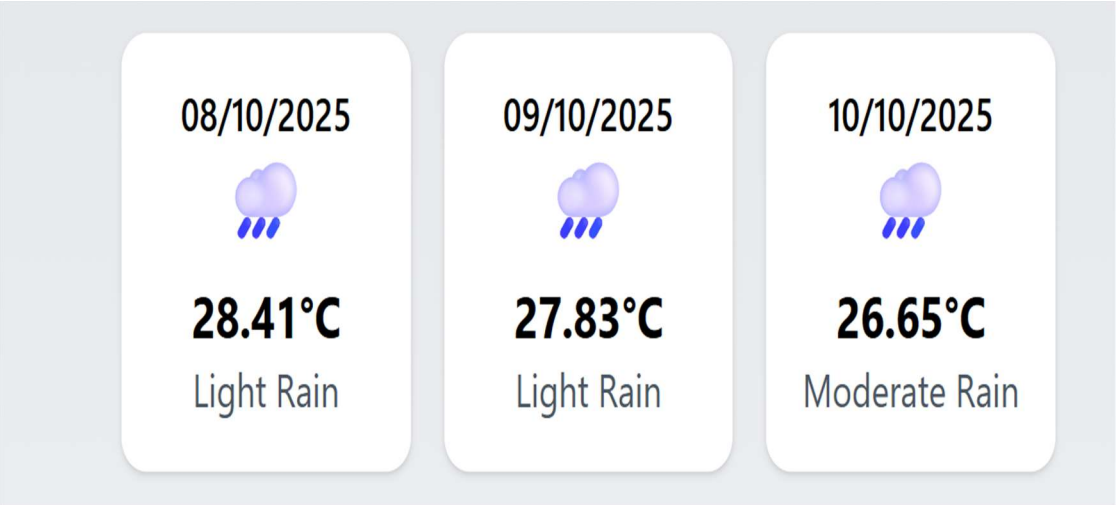
(You can insert your images here later)

1. **Homepage / Search Bar**

2. **Current Weather Display**



3. **3-Day Forecast Display**



4. **Error Message for Invalid City**

API Documentation

Current Weather API

- **Endpoint:**

  api.openweathermap.org/data/2.5/weather?q={city}&appid={apiKey}

- **What it does:** Gets the current weather for the city entered.
- **Key Parameters:**
  - q → city name (e.g., Chennai)
  - appid → your API key
  - units → metric (for °C)

3-Day Forecast API

- **Endpoint:**

  api.openweathermap.org/data/2.5/forecast?q={city}&appid={apiKey}

- **What it does:** Gets the forecast for the next few days.
- **Key Parameters:**
  - q → city name
  - appid → your API key
  - units → metric

5. Challenges & Solutions

1. **Invalid City Input**
   - **Challenge:** Users could type a wrong city name, which could break the app or give confusing results.
   - **Solution:** We added a friendly error message that clearly tells the user "City not found. Please try again."
2. **Forecast Data Too Detailed**
   - **Challenge:** The API provides weather data every 3 hours, which is too much information for a simple forecast.
   - **Solution:** We grouped the data by day and showed a **3-day forecast** with min/max temperatures and icons for clarity.
3. **Responsive Layout**
   - **Challenge:** The dashboard needed to look good on both desktop and mobile screens.
   - **Solution:** We adjusted the CSS and layout so that the dashboard is fully responsive and works well on all devices.
4. **Date Formatting**
   - **Challenge:** The API returns timestamps that are not user-friendly.
   - **Solution:** We converted the timestamps into **DD/MM/YYYY** format to make it easy to read.

etting Started with Create React App

This project was bootstrapped with Create React App.

Available Scripts

In the project directory, you can run:

```
npm start
```

Runs the app in the development mode.
Open http://localhost:3000 to view it in your browser.

The page will reload when you make changes.
You may also see any lint errors in the console.

`npm test`
Launches the test runner in the interactive watch mode.
See the section about running tests for more information.

`npm run build`
Builds the app for production to the `build` folder.
It correctly bundles React in production mode and optimizes the build for the best performance.
The build is minified and the filenames include the hashes.
Your app is ready to be deployed!

See the section about deployment for more information.

`npm run eject`
Note: this is a one-way operation. Once you `eject`, you can't go back!
If you aren't satisfied with the build tool and configuration choices, you can `eject` at any time. This command will remove the single build dependency from your project.
Instead, it will copy all the configuration files and the transitive dependencies (webpack, Babel, ESLint, etc) right into your project so you have full control over them. All of the commands except `eject` will still work, but they will point to the copied scripts so you can tweak them. At this point you're on your own.
You don't have to ever use `eject`. The curated feature set is suitable for small and middle deployments, and you shouldn't feel obligated to use this feature. However we understand that this tool wouldn't be useful if you couldn't customize it when you are ready for it.

Learn More

You can learn more in the Create React App documentation.

To learn React, check out the React documentation.

Code Splitting

This section has moved here: https://facebook.github.io/create-react-app/docs/code-splitting

Analyzing the Bundle Size

This section has moved here: https://facebook.github.io/create-react-app/docs/analyzing-the-bundle-size

Making a Progressive Web App

This section has moved here: https://facebook.github.io/create-react-app/docs/making-a-progressive-web-app

Advanced Configuration

This section has moved here: https://facebook.github.io/create-react-app/docs/advanced-configuration

Deployment

This section has moved here: https://facebook.github.io/create-react-app/docs/deployment

`npm run build` fails to minify

This section has moved here: https://facebook.github.io/create-react-app/docs/troubleshooting#npm-run-build-fails-to-minify

App.js

```
import { useState, useEffect } from "react";
import {
  LineChart,
  Line,
  XAxis,
  YAxis,
  Tooltip,
  ResponsiveContainer,
} from "recharts";


// Weather emoji mapping
const weatherEmoji = {
  Clear: "☀️",
  Clouds: "☁️",
```

```
  Rain: "🌧️",
  Snow: "❄️",
  Drizzle: "🌦️",
  Thunderstorm: "⛈️",
  Mist: "🌫️",
};

// Weather tips mapping
const weatherTips = {
  Clear: "It's sunny! Don't forget your sunglasses 😎",
  Clouds: "Cloudy skies today. You might need a light jacket 🌤️",
  Rain: "Take an umbrella! ☔",
  Snow: "Wear warm clothes! ❄️",
  Drizzle: "Carry a small umbrella 🌦️",
  Thunderstorm: "Stay indoors if possible ⛈️",
  Mist: "Drive carefully in the mist 🌫️",
};

// Animated effects for rain/snow
const WeatherEffect = ({ type }) => {
  if (type === "Rain" || type === "Drizzle") {
```

```
  return (
    <div className="absolute inset-0 pointer-events-none">
      {[...Array(30)].map((_, i) => (
        <div
          key={i}
          className="raindrop"
          style={{
            left: `${Math.random() * 100}%`,
            animationDelay: `${Math.random() * 2}s`,
            animationDuration: `${0.5 + Math.random()}s`,
          }}
        />
      ))}
    </div>
  );
} else if (type === "Snow") {
  return (
    <div className="absolute inset-0 pointer-events-none">
      {[...Array(30)].map((_, i) => (
        <div
          key={i}
          className="snowflake"
```

```
            style={{
              left: `${Math.random() * 100}%`,
              animationDelay: `${Math.random() * 5}s`,
              animationDuration: `${3 + Math.random() * 2}s`,
            }}
          />
        ))}
      </div>
    );
  }
  return null;
};

function App() {
  const [city, setCity] = useState("");
  const [weather, setWeather] = useState(null);
  const [forecast, setForecast] = useState(null);
  const [hourly, setHourly] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [favorites, setFavorites] = useState(
    JSON.parse(localStorage.getItem("favorites")) || []
```

```
  );

  const apiKey = "d4afe42722c98e631dfa248801d35a63";

  const fetchWeather = async (cityName) => {
    if (!cityName) return;
    setLoading(true);
    setError("");
    try {
      const resWeather = await fetch(
`https://api.openweathermap.org/data/2.5/weather?q=${cityName}&units=metric&appid=${apiKey}`
      );
      const dataWeather = await resWeather.json();
      if (dataWeather.cod !== 200) {
        setError("City not found. Try another one.");
        setWeather(null);
        setForecast(null);
        setHourly([]);
        setLoading(false);
        return;
      }
```

```
      setWeather(dataWeather);


      const resForecast = await fetch(

`https://api.openweathermap.org/data/2.5/forecast?q=${cityName}&units=metric&appid=${apiKey}`

      );

      const dataForecast = await resForecast.json();

      if (dataForecast.cod === "200") {

        setForecast(dataForecast);


        // Prepare hourly chart data (next 12 hours)

        const hourlyData = dataForecast.list.slice(0, 12).map((item) => ({

          time: item.dt_txt.split(" ")[1].slice(0, 5),

          temp: item.main.temp,

        }));

        setHourly(hourlyData);

      }


      // Voice reading

      const speech = `The current temperature in ${dataWeather.name} is ${dataWeather.main.temp} degrees Celsius with ${dataWeather.weather[0].description}`;
```

```
      const utter = new SpeechSynthesisUtterance(speech);
      window.speechSynthesis.speak(utter);
    } catch (err) {
      setError("Error fetching data.");
      setWeather(null);
      setForecast(null);
      setHourly([]);
      console.error(err);
    }
    setLoading(false);
  };

  const addFavorite = (cityName) => {
    if (!favorites.includes(cityName)) {
      const updated = [...favorites, cityName];
      setFavorites(updated);
      localStorage.setItem("favorites", JSON.stringify(updated));
    }
  };

  const removeFavorite = (cityName) => {
    const updated = favorites.filter((c) => c !== cityName);
```

```
    setFavorites(updated);
    localStorage.setItem("favorites", JSON.stringify(updated));
  };

  const useLocation = () => {
    if (!navigator.geolocation) return alert("Geolocation not
supported.");
    navigator.geolocation.getCurrentPosition(async (pos) => {
      const lat = pos.coords.latitude;
      const lon = pos.coords.longitude;
      setLoading(true);
      setError("");
      try {
        const resWeather = await fetch(

`https://api.openweathermap.org/data/2.5/weather?lat=${lat}&lon=${l
on}&units=metric&appid=${apiKey}`
        );
        const dataWeather = await resWeather.json();
        setCity(dataWeather.name);
        fetchWeather(dataWeather.name);
      } catch {
        setError("Cannot fetch location weather.");
```

```
    }
  });
};

const formatDate = (dt_txt) => {
  const parts = dt_txt.split(" ")[0].split("-");
  return `${parts[2]}/${parts[1]}/${parts[0]}`;
};

const getBackground = () => {
  if (!weather) return "from-blue-200 via-blue-100 to-white";
  const main = weather.weather[0].main;
  switch (main) {
    case "Clear":
      return "from-yellow-200 via-yellow-100 to-orange-100";
    case "Clouds":
      return "from-gray-300 via-gray-200 to-gray-100";
    case "Rain":
    case "Drizzle":
      return "from-blue-300 via-blue-200 to-blue-100";
    case "Snow":
      return "from-white via-blue-100 to-gray-100";
```

```
      case "Thunderstorm":
        return "from-gray-400 via-gray-300 to-gray-200";
      default:
        return "from-blue-200 via-blue-100 to-white";
    }
  };

  return (
    <div
      className={`min-h-screen p-4 relative bg-gradient-to-b
${getBackground()}`}
    >
      {/* Weather animation */}
      {weather && <WeatherEffect type={weather.weather[0].main} />}

      {/* Header */}
      <header className="text-center mb-8 relative z-10">
        <h1 className="text-4xl font-extrabold text-blue-800 mb-4
drop-shadow-md">
          Live Weather Dashboard
        </h1>
        <div className="flex flex-col sm:flex-row justify-center items-
center gap-2">
```

```
          <input
            type="text"
            placeholder="Enter city name"
            className="p-2 rounded border border-gray-400"
            value={city}
            onChange={(e) => setCity(e.target.value)}
          />
          <button
            className="p-2 bg-blue-500 text-white rounded shadow
hover:bg-blue-600 transition"
            onClick={() => fetchWeather(city)}
          >
            Get Weather
          </button>
          <button
            className="p-2 bg-green-500 text-white rounded shadow
hover:bg-green-600 transition"
            onClick={useLocation}
          >
            Use My Location
          </button>
        </div>
      </header>
```

```
      {/* Loading */}
      {loading && (
        <p className="text-center text-blue-700 font-semibold mt-4 relative z-10">
          Loading...
        </p>
      )}

      {/* Error */}
      {error && (
        <p className="text-center text-red-600 font-semibold mt-4 relative z-10">
          {error}
        </p>
      )}

      {/* Current Weather */}
      {weather && weather.main && !error && (
        <div className="max-w-md mx-auto bg-white rounded-xl shadow-lg p-6 text-center mb-6 relative z-10">
          <h2 className="text-xl font-semibold mb-2">{weather.name}</h2>
          <p className="text-5xl mb-2">
```

```
          {weatherEmoji[weather.weather[0].main] || "🌤"}
        </p>
        <p className="text-3xl font-bold">{weather.main.temp}°C</p>
        <p className="text-gray-600 capitalize">
          {weather.weather[0].description}
        </p>
        <div className="mt-4 bg-blue-100 p-3 rounded shadow">
          {weatherTips[weather.weather[0].main] || "Have a nice day!"}
        </div>
        <button
          className="mt-3 p-2 bg-yellow-400 rounded hover:bg-yellow-500 transition"
          onClick={() => addFavorite(weather.name)}
        >
          Add to Favorites
        </button>
      </div>
    )}

    {/* Favorite Cities */}
    {favorites.length > 0 && (
      <div className="max-w-md mx-auto mb-6">
```

```
        <h3 className="font-semibold mb-2 text-center">Favorite
Cities</h3>

        <div className="flex flex-wrap justify-center gap-2">

          {favorites.map((fav) => (

            <div

              key={fav}

              className="bg-white rounded shadow px-3 py-1 flex
items-center gap-2"

            >

              <button onClick={() => fetchWeather(fav)}>{fav}</button>

              <button

                className="text-red-500 font-bold"

                onClick={() => removeFavorite(fav)}

              >

                ×

              </button>

            </div>

          ))}

        </div>

      </div>

    )}



    {/* 3-Day Forecast */}
```

```
      {forecast?.list && !error && (
        <div className="max-w-md mx-auto mt-6 grid grid-cols-1
sm:grid-cols-3 gap-4 relative z-10">
          {forecast.list
            .filter((item, index) => index % 8 === 0)
            .slice(0, 3)
            .map((item, index) => (
              <div
                key={index}
                className="bg-white rounded-xl shadow p-4 text-center
transform transition hover:scale-105"
              >
                <p className="font-
semibold">{formatDate(item.dt_txt)}</p>
                <p className="text-3xl mb-2">
                  {weatherEmoji[item.weather[0].main] || "🌤"}
                </p>
                <p className="text-xl font-bold">{item.main.temp}°C</p>
                <p className="text-gray-600 capitalize">
                  {item.weather[0].description}
                </p>
              </div>
            ))}
```

```
        </div>
      )}

      {/* Hourly Forecast Chart */}
      {hourly.length > 0 && (
        <div className="max-w-md mx-auto mt-6 bg-white p-4 rounded shadow relative z-10">
          <h3 className="font-semibold mb-2 text-center">
            Next 12 Hours Forecast
          </h3>
          <ResponsiveContainer width="100%" height={200}>
            <LineChart data={hourly}>
              <XAxis dataKey="time" />
              <YAxis unit="°C" />
              <Tooltip />
              <Line
                type="monotone"
                dataKey="temp"
                stroke="#3b82f6"
                strokeWidth={2}
              />
            </LineChart>
```

```
    </ResponsiveContainer>
  </div>
)}

{/* CSS animations */}
<style jsx>{`
  .raindrop {
    position: absolute;
    width: 2px;
    height: 10px;
    background: rgba(255, 255, 255, 0.7);
    animation-name: fall;
    animation-timing-function: linear;
    animation-iteration-count: infinite;
  }
  @keyframes fall {
    0% {
      top: -10px;
    }
    100% {
      top: 100%;
    }
```

```
}
.snowflake {
  position: absolute;
  width: 6px;
  height: 6px;
  background: white;
  border-radius: 50%;
  opacity: 0.8;
  animation-name: snowFall;
  animation-timing-function: linear;
  animation-iteration-count: infinite;
}
@keyframes snowFall {
  0% {
    top: -10px;
    transform: translateX(0);
  }
  50% {
    transform: translateX(10px);
  }
  100% {
    top: 100%;
```

```
            transform: translateX(-10px);
          }
        }
      `}</style>
    </div>
  );
}

export default App;
```

Link:

https://github.com/SriMithuna/Live-Weather-Dashboard.git